Rineya Context Harness

Implementation Roadmap and Release Checklist

Companion to the Search-Reconstruct-Forget Development Plan

Version 1.0 | August 17, 2026

SMALL-MODEL-FIRST | LOCAL-FIRST | EVIDENCE-GROUNDED

Contents

1. Decision at a glance

2. Architecture in one page

3. Non-negotiable architecture rules

4. First model-facing tool catalog

5. Milestone sequence

- M0 - Specification freeze
- M1 - Deterministic state engine
- M2 - Rineya tool protocol
- M3 - Search and evidence ledger
- M4 - Recursive search-forget loop
- M5 - Ablation and opportunity frontier
- M6 - Real Rineya qualification
- M7 - Public Rineya Search demo
- M8 - Additional profiles

6. Technical validity checklist

7. RSF quality and efficiency measurements

- Quality
- Forgetting behavior
- Efficiency

8. Relative opportunity decision

9. Training sequence

10. Required regression cases

11. Public release manifest

12. Demo readiness checklist

13. Release boundaries

14. Final build order

15. References

> **Purpose:** A build-order document for implementing, qualifying, and releasing the Rineya Context Harness without losing the core recursive-state-forgetting research controls.
> **Primary model:** Rineya-1-454M. **Later model:** Rineya-1-Mini-344M.

> **Build rule:** Do not train the model to forget until the deterministic state engine can prove what was retained, deleted, restored, and required for success.

1. Decision at a glance

The first Rineya harness should remain a bounded, read-only research workflow:

```
Search -> Retrieve -> Inspect -> Reconstruct -> Update -> Delete
       -> Verify -> Rebuild Context -> Answer / Clarify / Abstain
```

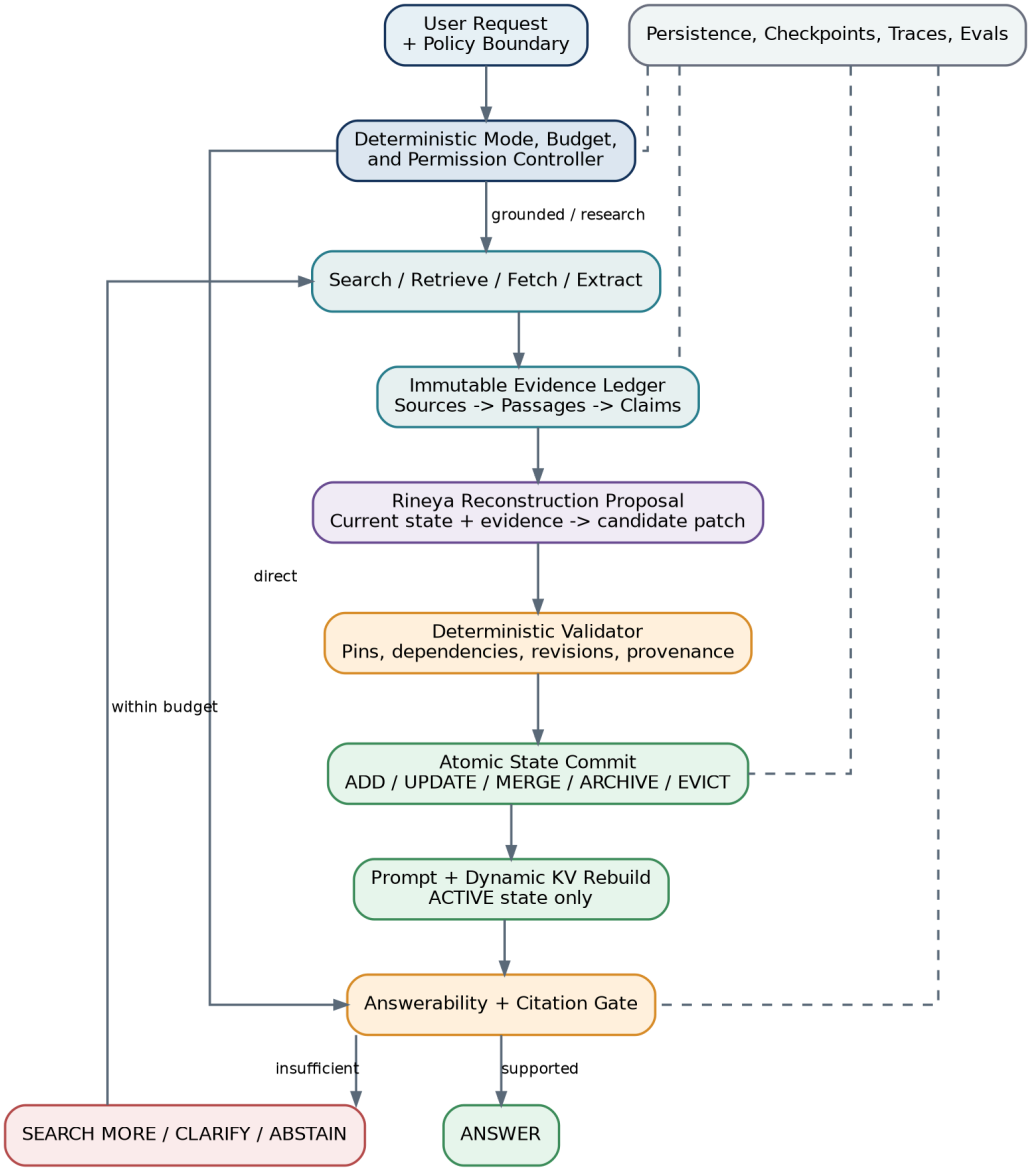
The model may propose semantic actions. Code owns:

- permissions;
- state revisions;
- idempotency;
- pinned invariants;
- dependencies;
- evidence provenance;
- budgets;
- deletion authorization;
- answerability;
- termination.

The defining rule is:

> **Rineya may propose what to forget; the harness must verify that forgetting does not destroy the minimal sufficient state.**

2. Architecture in one page



Rineya Context Harness architecture

The system maintains five distinct artifacts:

Artifact	Purpose	Normal model visibility
Active context	What Rineya sees now	Yes
Versioned state	What is currently true	Projected selectively
Evidence ledger	Immutable sources, passages, and claims	Selected records only
Trace	Connected execution history	No, except diagnostic summaries
Checkpoint	Recoverable validated revision	No

3. Non-negotiable architecture rules

1. **Stable IDs:** every memory and evidence record has an identifier.

2. **Reversible deletion:** model-facing deletion means archive or active-context eviction.
3. **No model purge:** permanent deletion is an explicit administrative boundary.
4. **Replacement first:** current state is replaced before old state leaves active context.
5. **Atomic commits:** update and delete operations succeed or fail together.
6. **Dependency guards:** unresolved goals, conflicts, citations, and dependents block unsafe deletion.
7. **Fresh prompt projection:** committed deletion triggers dynamic context/KV invalidation.
8. **Evidence over confidence:** grounded answers require claim-to-evidence support.
9. **Bounded loops:** every model, search, token, time, and retry budget is explicit.
10. **One model first:** use one accepted Rineya checkpoint before specialist agents.
11. **Six tools first:** keep the model tool surface small and semantic.
12. **Separate evaluator:** the evaluation harness remains independent of the agent runtime.

4. First model-facing tool catalog

Tool	Model intent	Code-owned enforcement
`get_state`	Read compact current state	projection and record limits
`search_web`	Resolve one evidence gap	provider, safe fetch, extraction, limits
`inspect_evidence`	Examine selected passages	immutable lookup and provenance
`update_memory`	Add, merge, correct, supersede	schema, revision, permissions, dependencies
`delete_memory`	Archive or evict obsolete state	pins, replacements, provenance, idempotency
`finish`	Answer, clarify, partially answer, or abstain	answerability and citation gate

A successful deletion returns `prompt_rebuild_required: true`.

5. Milestone sequence



Development roadmap

M0 - Specification freeze

Implement or finalize

- terminology;
- schemas;
- six-tool contracts;
- state lifecycle;
- delete, archive, restore, and purge boundaries;
- workflow graph;
- threat model;
- evaluation protocol;
- configuration schema.

Gate

- examples validate;
- no ambiguous destructive operation;
- all versions fixed in one manifest.

M1 - Deterministic state engine

Implement

- SQLite WAL or equivalent local store;
- ordered migrations;
- stable IDs;
- optimistic revisions;
- semantic idempotency;
- pins and dependencies;
- atomic state patches;
- archive, eviction, restore;
- checkpoints and rollback;
- append-only transition events.

Gate

- pinned invariant deletion is structurally impossible;
- stale revisions are rejected;
- rollback exactly reproduces prior state;
- no state corruption across forced failures.

M2 - Rineya tool protocol

Implement

- model backend interface;
- strict schemas;
- parser and validator;
- one bounded repair attempt;
- deterministic fixture backend;
- OpenAI-compatible local endpoint;
- Ollama-compatible endpoint if needed.

Measure

- valid tool-name rate;
- valid schema rate;
- stable-ID use;
- correct state revision use;
- correct finish outcome;
- repair success rate.

Gate

- all failures are visible and categorized;
- invalid operations are never silently coerced.

M3 - Search and evidence ledger

Implement

- search provider interface;
- safe fetcher;
- extraction;
- segmentation;
- deduplication;
- passage ranking and source diversity;
- immutable source/passage/claim records;
- citation verifier;
- prompt-injection marking.

Gate

- offline fixture search passes end to end;
- evidence provenance is reproducible;
- unrelated citations fail verification.

M4 - Recursive search-forget loop

Implement

```
identify gap
-> search
-> inspect evidence
-> reconstruct current state
-> add replacements
-> archive/evict obsolete state
-> rebuild context
-> verify answerability
-> answer or continue
```

Gate

- every loop is bounded;
- context savings are measured;
- restore works after a controlled premature deletion;
- no answer bypasses the gate.

M5 - Ablation and opportunity frontier

Run the frozen comparison:

Condition	Strategy
A	Full history
B	Sliding window
C	One-shot summary
D	Structured state, no deletion
E	Structured state plus explicit deletion

Condition	Strategy
F	Deletion plus archive and restore
G	Oracle recursive-state forgetting

Also compare:

```
direct model
raw RAG
one-shot compressed RAG
structured evidence without deletion
full Search-Reconstruct-Forget
```

Gate

- reports include quality, context tokens, latency, deletion regret, future-utility recall, and citation behavior;
- selection uses proportional opportunity and Pareto evidence;
- sealed tasks remain untouched.

M6 - Real Rineya qualification

Connect Rineya-1-454M with:

- exact tokenizer;
- exact chat template;
- thinking disabled;
- frozen adapter scale;
- deterministic decoding baseline.

Then qualify Rineya-1-Mini-344M separately.

Gate

- tool protocol measured on a frozen set;
- original chat and RAG capabilities are preserved proportionally;
- state-reduction behavior is compared with deterministic/oracle policies.

M7 - Public Rineya Search demo

Show:

```
sources fetched
passages extracted
facts retained
items updated
items evicted
pinned invariants
context before/after
citation verification
final concise answer
```

Gate

- human review completed;
- model and harness versions stated separately;

- public claims match reproducible traces.

M8 - Additional profiles

Only after the search profile is stable:

- local documents and PDFs;
- SWE-bench/Terminal-Bench code profile;
- scoped persistent user memory;
- MCP boundary;
- adapter specialization;
- model escalation;
- side-effecting tools with approvals.

6. Technical validity checklist

These failures invalidate an experiment rather than counting as ordinary quality loss:

- ▢ state serialization is valid;
- ▢ model and tokenizer are compatible;
- ▢ required output is present;
- ▢ no NaN or incomplete results;
- ▢ evidence provenance can be reproduced;
- ▢ pinned invariants were never deleted;
- ▢ database migrations completed;
- ▢ run manifest identifies all versions;
- ▢ sealed evaluation data was not used for tuning.

7. RSF quality and efficiency measurements

Quality

- task success;
- next-action correctness;
- final-answer correctness;
- supported-answer correctness;
- abstention correctness;
- procedure correctness;
- invariant retention;
- contradiction resolution;
- citation precision and coverage;
- human conversational naturalness.

Forgetting behavior

- future-utility recall;
- obsolete-eviction precision;

- obsolete-eviction recall;
- deletion regret;
- restore frequency;
- dependency violations;
- state-reconstruction accuracy.

Efficiency

- original raw tokens;
- active tokens before compaction;
- active tokens after compaction;
- stable prefix tokens;
- prefill latency;
- rebuild latency;
- total model calls;
- total search calls;
- end-to-end latency;
- peak memory.

8. Relative opportunity decision

Use a resource-versus-quality comparison:

```
context_gain = max(0, 1 - candidate_active_tokens / reference_active_tokens)
quality_loss = weighted normalized candidate-vs-reference losses
opportunity_margin = context_gain - quality_loss
```

A candidate is scientifically interesting when it produces a positive, robust opportunity margin without technical invalidity. Report multiple weight policies and the Pareto frontier rather than hiding all judgment in one scalar.

9. Training sequence

1. Deterministic state engine and oracle policy.
2. Prompt-only tool compliance.
3. SFT for valid state and delete transactions.
4. DPO for premature deletion, failure to delete, and unsupported finish behavior.
5. Long trajectory evaluation.
6. Optional RL/GRPO only after deterministic and supervised baselines are stable.

Do not reward deletion volume. Reward task outcome, valid state, preserved invariants, and efficient context only when quality remains supported.

10. Required regression cases

- [ ] restaurant route A -> B,C;
- [ ] Tower of Hanoi current-state replacement;
- [ ] same current state from different histories;

- [ ] minimally edited state that invalidates old memory;
- [ ] deleted item later becomes useful;
- [ ] changed goal after compaction;
- [ ] cycle prevention requiring compact visited-state hashes;
- [ ] delete pinned invariant attempt;
- [ ] delete unresolved conflict attempt;
- [ ] delete evidence without supported replacement;
- [ ] stale revision;
- [ ] idempotency-key conflict;
- [ ] prompt injection in fetched text;
- [ ] private-network URL;
- [ ] unrelated citation;
- [ ] budget exhaustion;
- [ ] successful targeted restore.

11. Public release manifest

Every published harness result should identify:

```
model checkpoint and hash
adapter checkpoint and hash
adapter dose
model runtime
harness version and commit
tool schema version
state schema version
evaluation suite version
search provider or fixture
context budget
search/model-call budgets
decoding configuration
result artifacts and trace bundle
```

12. Demo readiness checklist

- [ ] current question genuinely requires external evidence;
- [ ] sources are visible and attributable;
- [ ] active state shows goals, facts, gaps, and conflicts;
- [ ] at least one state update is understandable to viewers;
- [ ] at least one safe eviction is visible;
- [ ] source evidence remains recoverable;
- [ ] before/after token counts are displayed;
- [ ] answerability gate is shown;
- [ ] final response remains concise and natural;
- [ ] demo uses a frozen reproducible run;
- [ ] no claim exceeds measured evidence.

13. Release boundaries

The first harness release does **not** claim:

- universal autonomy;
- permanent personal memory;
- model-native proof of optimal forgetting;
- browser or desktop control;
- coding-agent competence;
- superiority over RAG without ablation evidence;
- safety for side-effecting tools.

It does claim an inspectable architecture for testing whether explicit state reconstruction and reversible deletion improve small-model reasoning under limited context.

14. Final build order

```
schemas
-> deterministic state engine
-> delete and restore contracts
-> evidence ledger
-> context rebuild
-> six-tool model protocol
-> bounded search workflow
-> answerability verification
-> RSF ablations
-> real Rineya qualification
-> public demo
```

> **Do not let web search become a shortcut around the core research question.** Search supplies information; recursive-state forgetting determines how that information is maintained, replaced, removed, verified, and presented to a small model over time.

15. References

1. *Short-Response Model - Research and Engineering Blueprint*, Version 1.0, July 2026.
2. Rong Wu et al., *MemHarness: Memory Is Reconstructed, Not Replayed*, arXiv:2607.28272, 2026.
3. *AI Agent Systems From First Principles*, 2026.
4. *Rineya Context Harness v0.1.0 Architecture Specification and Repository Skeleton*, August 2026.